COMPULIN

Equipment Hire & Rental Management System

Finalized Database Table Specification - Updated After Client Clarification

Basis of this revision. This version updates the previous 17-table design using the client's latest answers. The biggest changes are: company onboarding is done manually by the Compulin Admin; there is no online registration request/approval/verification workflow; login username is the user's email and the password is shared manually through WhatsApp (no WhatsApp integration); renter identity documents must be checked and a record/copy retained; inventory and rental lines must support quantities; the refundable deposit is calculated during item selection; and lost/non-returned quantities must be recordable.

AWS: the client selected AWS for deployment/hosting. This affects deployment architecture, not the logical MySQL table structure, so no special AWS table is required.

Final recommendation: use **18 core tables**. The previous design had 17 tables; one new table, **customer_documents**, is added for ID/driving licence/passport records and copies. Several existing tables are simplified or changed to match the manual workflow and quantity-based inventory.

1. What Changed From the Previous Design

Area	Final change
companies	Remove the online request/approval idea. A Compulin Admin creates the company record after the company registers in person. PENDING/REJECTED are no longer needed for normal onboarding.
users	Email remains the login username. Store only a secure password hash. The initial password may be communicated manually through WhatsApp, but the database does not store a WhatsApp message or integration record.
customers	Renter registration is performed by staff. Customer/renter remains business data, not a login actor.
customer_documents - NEW	Stores the checked NIC/ID, driving licence and/or passport details plus the saved copy/path of the document.
equipment	Change from only individually tracked assets to a quantity-aware item registration model. Store total quantity and available quantity, because the client explicitly requires issued quantities and lost-item quantities.
rental_items	Add quantity and deposit snapshot fields. This supports the item-selection screen and lets the system calculate the refundable deposit before issue.
returns / return_items	Keep an item-return screen and record the quantity actually returned for each rental line.
damage_records	Record damaged quantity as well as assessment and charge details.
lost_items	Use this as the client's 'Lost Note' data. Store the quantity lost/non-returned and the action/status of the case.
registration/verification tables	Do not create company_registration_requests, company_verification, approval, OTP or verification-document tables for company onboarding.

Module 1 - Company, Users & Inventory

companies

Purpose: One row represents one rental company that has been manually registered in person by the Compulin Admin.

Attribute	Type	Key / Constraint	Meaning / Use
company_id	INT 4 digit	PK, AUTO_INCREMENT	Unique company ID.
company_name	VARCHAR(150)	NOT NULL	Rental company name.
registration_no	VARCHAR(60)	NULL/UNIQUE if supplied	Business registration reference if the company provides one. It is stored as information, not used for an online verification workflow.
email	VARCHAR(120)	NOT NULL	Main company contact email.
phone	VARCHAR(25)		Main contact number.
address	VARCHAR(255)		Business address.
registered_by	INT	FK -> users.user_id	Compulin Admin who manually registered the company.
registration_date	DATETIME	NOT NULL	When the company was entered into the system.
status	VARCHAR(20)	NOT NULL	ACTIVE, INACTIVE or SUSPENDED.

Design/update note: UPDATED: remove PENDING/REJECTED approval states from normal onboarding. No online request or verification table is needed.

users

Purpose: Stores system login accounts for Compulin Admin, Rental Company Admin and Rental Officer.

Attribute	Type	Key / Constraint	Meaning / Use
user_id	INT	PK, AUTO_INCREMENT	Unique user ID.
company_id	INT	FK -> companies.company_id, NULL	NULL for Compulin Admin; company ID for company staff.
full_name	VARCHAR(120)	NOT NULL	Staff member name.
email	VARCHAR(120)	UNIQUE, NOT NULL	Login username, as specified by the client.
password_hash	VARCHAR(255)	NOT NULL	Hashed password only. Never store the plain password.
phone	VARCHAR(25)		Contact number.
role	VARCHAR(30)	NOT NULL	COMPULIN_ADMIN, COMPANY_ADMIN, RENTAL_OFFICER.
status	VARCHAR(20)	NOT NULL	ACTIVE or INACTIVE.
created_at	DATETIME	NOT NULL	Account creation time.

Design/update note: UPDATED: no OTP, email-verification or WhatsApp-integration fields. The password can be sent manually by the Compulin Admin through WhatsApp outside the system.

Change password - rentee admin

equipment_categories

Purpose: Groups registered items into categories.

Attribute	Type	Key / Constraint	Meaning / Use
category_id	INT	PK, AUTO_INCREMENT	Unique category ID.

Attribute	Type	Key / Constraint	Meaning / Use
company_id	INT	FK -> companies.company_id, NOT NULL	Company that owns the category.
category_name	VARCHAR(100)	NOT NULL	Category name.
description	VARCHAR(255)		Optional description.
status	VARCHAR(20)	NOT NULL	ACTIVE or INACTIVE.

Design/update note: Kept because it avoids repeating category names and supports inventory filtering.

equipment

Purpose: Item registration/master inventory table. One row can represent an item/product type with a stock quantity.

Attribute	Type	Key / Constraint	Meaning / Use
equipment_id	INT	PK, AUTO_INCREMENT	Unique inventory item ID.
company_id	INT	FK -> companies.company_id, NOT NULL	Rental company that owns the item.
category_id	INT	FK -> equipment_categories.category_id	Category.
item_name	VARCHAR(150)	NOT NULL	Name shown on the item registration/selection screen.
item_code	VARCHAR(80)	UNIQUE within company recommended	Internal stock/item code.
description	VARCHAR(500)		Item details.
rental_rate	DECIMAL(12,2)	NOT NULL	Standard rental price per unit.
rate_period	VARCHAR(20)	NOT NULL	DAILY, WEEKLY, MONTHLY etc.
refundable_deposit_per_unit	DECIMAL(12,2)	DEFAULT 0	Deposit used when calculating the refundable deposit during item selection.
total_quantity	INT	NOT NULL	Total units owned/registered.
available_quantity	INT	NOT NULL	Units currently available for new rentals.
status	VARCHAR(30)	NOT NULL	ACTIVE, INACTIVE or OUT_OF_SERVICE.
created_at	DATETIME	NOT NULL	Item registration time.

security deposit

inactive - if everything is out of stock

Design/update note: MAJOR UPDATE: quantity is now explicit. This better matches the client's requirement to issue items with quantity and enter the volume/number of lost items.

derived attr. don't put in er diagram

Module 2 - Renter & Rental Management

customers

Purpose: Stores the rentee/customer registered by staff. The renter does not log in.

Attribute	Type	Key / Constraint	Meaning / Use
customer_id	INT	PK, AUTO_INCREMENT	Unique renter ID.
company_id	INT	FK -> companies.company_id, NOT NULL	Rental company that owns the renter record.
customer_name	VARCHAR(150)	NOT NULL	Renter name.
email	VARCHAR(120)		Contact email.
phone	VARCHAR(25)	NOT NULL recommended	Primary contact number.
address	VARCHAR(255)		Address.
customer_type	VARCHAR(20)	NOT NULL	INDIVIDUAL or BUSINESS.
status	VARCHAR(20)	NOT NULL	ACTIVE, INACTIVE or BLOCKED. BLOCKED if going to the police doc side
created_by	INT	FK -> users.user_id	Officer who registered the renter.
created_at	DATETIME	NOT NULL	Registration time.

Design/update note: UPDATED: identity-document details are moved to customer_documents so multiple documents can be recorded cleanly.

customer_documents

Purpose: NEW table for the client's requirement to check ID/driving licence/passport and keep a record and copy.

Attribute	Type	Key / Constraint	Meaning / Use
document_id	INT	PK, AUTO_INCREMENT	Unique document record.
customer_id	INT	FK -> customers.customer_id, NOT NULL	Renter whose document was checked.
document_type	VARCHAR(30)	NOT NULL	NIC_ID, DRIVING_LICENCE, PASSPORT or OTHER.
document_number	VARCHAR(80)	NOT NULL	Document number/reference.
document_copy_path	VARCHAR(500)	NOT NULL recommended	Stored file/object path or URL to the retained copy. Do not store the image directly in a normal VARCHAR. View ID copy option
expiry_date	DATE	NULL	Useful for licence/passport if applicable.
checked_by	INT	FK -> users.user_id, NOT NULL	Staff member who checked it.
checked_at	DATETIME	NOT NULL	When it was checked.
notes	VARCHAR(255)		Optional notes.

Design/update note: NEW: this is the clearest database representation of the client's explicit identity-checking requirement.

rentals

Purpose: Header for one rental transaction.

Attribute	Type	Key / Constraint	Meaning / Use
rental_id	INT	PK, AUTO_INCREMENT	Unique rental ID.

Attribute	Type	Key / Constraint	Meaning / Use
company_id	INT	FK -> companies.company_id, NOT NULL	Company conducting the rental.
customer_id	INT	FK -> customers.customer_id, NOT NULL	Renter.
created_by	INT	FK -> users.user_id, NOT NULL	Rental Officer.
rental_date	DATETIME	NOT NULL	Record creation time.
start_date	DATE	NOT NULL	Issue/start date.
due_date	DATE	NOT NULL	Agreed return date.
rental_status	VARCHAR(30)	NOT NULL	DRAFT, ACTIVE, OVERDUE, PARTIALLY_RETURNED, RETURNED, CLOSED, CANCELLED. DRAFT once rental is made until start date
notes	VARCHAR(500)		General notes.
created_at	DATETIME	NOT NULL	Database timestamp.

Design/update note: UPDATED: total deposit is better derived from rental_items and recorded as actually received in deposits, avoiding duplicated sources of truth.

rental_items

Purpose: Stores each selected inventory item and quantity for a rental.

Attribute	Type	Key / Constraint	Meaning / Use
rental_item_id	INT	PK, AUTO_INCREMENT	Unique rental line.
rental_id	INT	FK -> rentals.rental_id, NOT NULL	Parent rental.
equipment_id	INT	FK -> equipment.equipment_id, NOT NULL	Selected item.
quantity	INT	NOT NULL	Number of units issued/selected.
rate_per_unit	DECIMAL(12,2)	NOT NULL	Rate copied at rental time.
rate_period	VARCHAR(20)	NOT NULL FK, from table 'equipment'	Rate basis.
deposit_per_unit	DECIMAL(12,2)	DEFAULT 0	Deposit copied at rental time.
line_deposit	DECIMAL(12,2)	NOT NULL	quantity x deposit_per_unit; snapshot for audit.
item_status	VARCHAR(30)	NOT NULL	SELECTED, ISSUED, PARTIALLY_RETURNED, RETURNED, LOST, CLOSED.
issued_at	DATETIME	NULL	Actual handover time.

Design/update note: MAJOR UPDATE: quantity and deposit snapshot fields directly support the client's item-selection and refundable-deposit requirements.

rental_extensions

Purpose: Preserves due-date changes if an existing rental is extended.

Attribute	Type	Key / Constraint	Meaning / Use
extension_id	INT	PK, AUTO_INCREMENT	Unique extension.
rental_id	INT	FK -> rentals.rental_id, NOT NULL	Rental.
old_due_date	DATE	NOT NULL	Previous due date.
new_due_date	DATE	NOT NULL	New agreed due date.

Attribute	Type	Key / Constraint	Meaning / Use
extension_charge	DECIMAL(12,2)	DEFAULT 0	Extra charge if applicable.
approved_by	INT	FK -> users.user_id, NOT NULL	Staff who recorded it.
reason	VARCHAR(255)		Reason.
created_at	DATETIME	NOT NULL	Timestamp.

Design/update note: Kept from the earlier requirements because late/extended rentals still need history; this does not conflict with the new client answers.

customer_secondary_contacts

Attribute	Type	Key / Constraint	Meaning / Use
`secondary_contact_id`	INT	PK, AUTO_INCREMENT	Unique secondary-contact record
`customer_id`	INT	FK → `customers.customer_id`, NOT NULL	Customer this contact belongs to
`contact_name`	VARCHAR(100)	NOT NULL	Secondary person's name
`relationship`	VARCHAR(50)	NULL	Relationship to customer, e.g. parent, sibling, friend
`phone_number`	VARCHAR(20)	NOT NULL	Main contact number
`alternate_phone`	VARCHAR(20)	NULL	Optional additional number
`email`	VARCHAR(150)	NULL	Optional email address
`address`	VARCHAR(255)	NULL	Optional address
`notes`	VARCHAR(255)	NULL	Any relevant notes

Module 3 - Billing, Deposit & Payment

invoices

Purpose: Stores bills generated for a rental.

Attribute	Type	Key / Constraint	Meaning / Use
invoice_id	INT	PK, AUTO_INCREMENT	Unique invoice.
rental_id	INT	FK -> rentals.rental_id, NOT NULL	Rental being billed.
invoice_date	DATE	NOT NULL	Invoice date.
due_date	DATE		Payment due date.
subtotal	DECIMAL(12,2)	NOT NULL	Base amount.
additional_charges	DECIMAL(12,2)	DEFAULT 0	Extra charges.
total_amount	DECIMAL(12,2)	NOT NULL	Final billed amount.
amount_paid	DECIMAL(12,2)	DEFAULT 0	Amount paid. amount paid > 0 AND < total_amount ~PARTIALLY_PAID
balance_due	DECIMAL(12,2)	NOT NULL	Outstanding balance.
status	VARCHAR(25)	NOT NULL	UNPAID, PARTIALLY_PAID, PAID, CANCELLED.

Design/update note: Kept. The client did not request an online payment gateway.

charges

Purpose: Traceable individual charges for rental, late return, damage, loss, etc.

Attribute	Type	Key / Constraint	Meaning / Use
charge_id	INT	PK, AUTO_INCREMENT	Unique charge.
rental_id	INT	FK -> rentals.rental_id, NOT NULL	Rental.
rental_item_id	INT	FK -> rental_items.rental_item_id, NULL	Specific rental line if applicable.
invoice_id	INT	FK -> invoices.invoice_id, NULL	Invoice containing the charge.
charge_type	VARCHAR(30)	NOT NULL	RENTAL, EXTENSION, LATE, DAMAGE, LOST_ITEM, OTHER.
description	VARCHAR(255)		Reason.
amount	DECIMAL(12,2)	NOT NULL	Charge amount.
charge_date	DATETIME	NOT NULL	Created time.
created_by	INT	FK -> users.user_id, NOT NULL	Staff user.

Design/update note: Kept because damage/lost/late charges need a clear audit trail.

payments

Purpose: Stores each ordinary payment against an invoice.

Attribute	Type	Key / Constraint	Meaning / Use
payment_id	INT	PK, AUTO_INCREMENT	Unique payment.
invoice_id	INT	FK -> invoices.invoice_id, NOT NULL	Invoice.
amount	DECIMAL(12,2)	NOT NULL	Payment amount.
payment_date	DATETIME	NOT NULL	When received.

Attribute	Type	Key / Constraint	Meaning / Use
payment_method	VARCHAR(30)	NOT NULL	CASH, CARD, BANK_TRANSFER etc.
reference_no	VARCHAR(100)		Reference if available.
received_by	INT	FK -> users.user_id, NOT NULL	Staff member.
status	VARCHAR(20)	NOT NULL	COMPLETED or VOIDED.

but this system doesn't integrate online payments

Design/update note: No online gateway/integration table is required.

deposits

Purpose: Tracks the refundable deposit collected from the renter and its later refund/deduction.

Attribute	Type	Key / Constraint	Meaning / Use
deposit_id	INT	PK, AUTO_INCREMENT	Unique deposit record.
rental_id	INT	FK -> rentals.rental_id, UNIQUE recommended	Rental.
calculated_deposit	DECIMAL(12,2)	NOT NULL	Deposit calculated from selected rental_items.
deposit_amount_received	DECIMAL(12,2)	NOT NULL	Actual deposit collected from the renter.
amount_deducted	DECIMAL(12,2)	DEFAULT 0	Amount retained for approved charges.
amount_refunded	DECIMAL(12,2)	DEFAULT 0	Amount returned.
received_date	DATETIME		Collection time.
refund_date	DATETIME	NULL	Refund time.
received_by	INT	FK -> users.user_id	Staff who collected it.
status	VARCHAR(25)	NOT NULL	PENDING, HELD, PARTIALLY_REFUNDED, REFUNDED, FORFEITED.

DERIVED ATTR from deposit_amount_received and amount_refunded

Design/update note: UPDATED: explicitly separates the system-calculated deposit from the amount actually collected, matching the client's item-selection/deposit requirement.

Module 4 - Return, Damage, Lost Items & Settlement

returns

Purpose: Header for each item-return event.

Attribute	Type	Key / Constraint	Meaning / Use
return_id	INT	PK, AUTO_INCREMENT	Unique return event.
rental_id	INT	FK -> rentals.rental_id, NOT NULL	Rental.
return_date	DATETIME	NOT NULL	Return date/time.
processed_by	INT	FK -> users.user_id, NOT NULL	Officer processing return.
return_type	VARCHAR(20)	NOT NULL	FULL or PARTIAL.
notes	VARCHAR(500)		General return notes.

Design/update note: Matches the client's required Item Return Screen.

return_items

Purpose: Records which rental lines came back and how many units were returned.

Attribute	Type	Key / Constraint	Meaning / Use
return_item_id	INT	PK, AUTO_INCREMENT	Unique returned line.
return_id	INT	FK -> returns.return_id, NOT NULL	Return event.
rental_item_id	INT	FK -> rental_items.rental_item_id, NOT NULL	Original rental line.
quantity_returned	INT	NOT NULL	Number of units returned in this event.
condition_status	VARCHAR(30)	NOT NULL	GOOD, DAMAGED, MISSING_PARTS, NEEDS_MAINTENANCE.
inspection_notes	VARCHAR(500)		Inspection details.
returned_at	DATETIME	NOT NULL	Actual receipt time.

Design/update note: MAJOR UPDATE: quantity_returned allows partial quantity returns, not only item-by-item serialised returns.

damage_records

Purpose: Stores damage assessment after returned items are inspected.

Attribute	Type	Key / Constraint	Meaning / Use
damage_id	INT	PK, AUTO_INCREMENT	Unique damage record.
return_item_id	INT	FK -> return_items.return_item_id, NOT NULL	Returned line.
damaged_quantity	INT	NOT NULL	Number of returned units affected.
damage_description	VARCHAR(500)	NOT NULL	Observed damage.
damage_level	VARCHAR(20)	NOT NULL	MINOR, MODERATE, SEVERE.
estimated_cost	DECIMAL(12,2)	DEFAULT 0	Estimated cost.
final_charge	DECIMAL(12,2)	DEFAULT 0	Approved renter charge.
assessed_by	INT	FK -> users.user_id, NOT NULL	Staff assessor.
assessment_date	DATETIME	NOT NULL	Assessment time.
status	VARCHAR(30)	NOT NULL	ASSESSED, CHARGED, REPAIRED, WAIVED.

Design/update note: UPDATED: damaged_quantity supports quantity-based stock.

lost_items

Purpose: Client's Lost Note record for lost, stolen or never-returned quantities.

Attribute	Type	Key / Constraint	Meaning / Use
lost_item_id	INT	PK, AUTO_INCREMENT	Unique lost-note line.
rental_item_id	INT	FK -> rental_items.rental_item_id, NOT NULL	Issued rental line.
quantity_lost	INT	NOT NULL	Number/volume of units lost or not returned.
loss_type	VARCHAR(30)	NOT NULL	LOST, STOLEN, NON_RETURNED.
reported_date	DATETIME	NOT NULL	When the lost note was created.
reason	VARCHAR(500)		Case details.
replacement_cost_per_unit	DECIMAL(12,2)	DEFAULT 0	Replacement value per unit.
charge_amount	DECIMAL(12,2)	DEFAULT 0	Final charge for the lost quantity.
reported_by	INT	FK -> users.user_id, NOT NULL	Officer recording the lost note.
status	VARCHAR(30)	NOT NULL	OPEN, RECOVERED, CHARGED, SETTLED.
notes	VARCHAR(500)		Additional follow-up notes.

Design/update note: MAJOR UPDATE: quantity_lost directly implements the client's instruction that the Lost Note screen must allow entry of the number/volume of lost items. No police-report table is added because the latest response did not explicitly require that workflow.

settlements

Purpose: Final financial closure snapshot for a rental.

Attribute	Type	Key / Constraint	Meaning / Use
settlement_id	INT	PK, AUTO_INCREMENT	Unique settlement.
rental_id	INT	FK -> rentals.rental_id, UNIQUE recommended	Rental.
rental_charges	DECIMAL(12,2)	DEFAULT 0	Final base rental charges.
late_charges	DECIMAL(12,2)	DEFAULT 0	Late charges.
damage_charges	DECIMAL(12,2)	DEFAULT 0	Damage charges.
lost_item_charges	DECIMAL(12,2)	DEFAULT 0	Lost/non-returned charges.
total_charges	DECIMAL(12,2)	NOT NULL	Final total.
deposit_used	DECIMAL(12,2)	DEFAULT 0	Deposit applied.
deposit_refunded	DECIMAL(12,2)	DEFAULT 0	Deposit refunded.
final_balance	DECIMAL(12,2)	NOT NULL	Remaining amount.
settled_at	DATETIME		Closure time.
settled_by	INT	FK -> users.user_id, NOT NULL	Staff user.
status	VARCHAR(20)	NOT NULL	PENDING or SETTLED.

Design/update note: Kept to give a clean final audit record after returns, damage/loss charges and deposit handling.

5. Final Relationship Map for the ERD

• companies 1:M users
• companies 1:M equipment_categories
• companies 1:M equipment
• companies 1:M customers
• companies 1:M rentals
• equipment_categories 1:M equipment
• customers 1:M customer_documents
• customers 1:M rentals
• rentals 1:M rental_items
• equipment 1:M rental_items over time
• rentals 1:M rental_extensions
• rentals 1:M invoices
• rentals 1:M charges
• invoices 1:M payments
• rentals 1:0..1 deposits
• rentals 1:M returns
• returns 1:M return_items
• rental_items 1:M return_items over multiple partial returns
• return_items 1:0..M damage_records
• rental_items 1:0..M lost_items
• rentals 1:0..1 settlements
• users 1:M operational records through registered_by / created_by / checked_by / received_by / processed_by / assessed_by / reported_by / settled_by

6. How the Updated Database Works Through the Client's MVP

Stage	Database action
1. Manual company registration	The rental company visits/contacts Compulin in person. The Compulin Admin directly creates companies and user records. There is no request, approval or verification queue.
2. Credentials	The staff user's email is the username. A password is created securely and stored only as password_hash. The Compulin Admin can communicate the initial password manually through WhatsApp; the application does not integrate with WhatsApp.
3. Item registration	The Company Admin registers equipment/item types and stock quantities. total_quantity and available_quantity control stock.
4. Rentee registration	The Rental Officer creates the customer record, checks NIC/ID, driving licence and/or passport, and records the document plus its retained copy in customer_documents.
5. Item selection	The officer selects equipment and enters quantity. rental_items stores the transaction-time rate and deposit per unit.
6. Refundable deposit calculation	The system calculates line_deposit = quantity x deposit_per_unit and totals the selected lines. deposits stores the calculated amount and the amount actually collected.
7. Issue	When items are handed over, the rental becomes ACTIVE, rental_items become ISSUED and available_quantity is reduced.
8. Return	A returns record is created. return_items records the quantity returned and its condition.
9. Damage	Damaged quantities are assessed in damage_records. A DAMAGE charge may be generated.
10. Lost/stolen/non-returned	A lost_items Lost Note is created with quantity_lost. A LOST_ITEM charge may be generated and stock is adjusted appropriately.
11. Settlement	Payments, charges and deposit deductions/refunds are reconciled. settlements stores the final closure snapshot.

7. Tables You Should NOT Add

Based on the final clarification, do **not** add separate tables for online company registration requests, company approval/rejection, company verification, OTP verification, WhatsApp messages/integration, customer login accounts, or a generic reports table. Also, do not add a police-report table unless the client later explicitly asks the system to create/manage police complaint records.

8. Important Implementation Rules

• Use DECIMAL for money.
• Use positive quantity checks: total_quantity >= 0, available_quantity >= 0, rental_items.quantity > 0, quantity_returned > 0 and quantity_lost > 0.
• Never allow selected/issued quantity to exceed available_quantity.
• Never allow total returned + lost quantity for a rental line to exceed the issued quantity.
• Keep transaction-time rate and deposit values in rental_items even if equipment prices later change.
• Keep document copies in protected file/object storage and store only the path/key in customer_documents.
• Keep company data isolated using company_id and validate that customers, users and equipment used in a rental belong to the same company.
• Generate reports from transactional tables instead of storing a generic report row.
• AWS is the hosting target; the same logical schema can be implemented in MySQL and deployed using an AWS-managed or self-hosted database setup.